

Assignment – 16.1

2303A51860

Task 1 – Student Information System Schema

PROMPT :

--create tables Students, Courses, Enrollments, primary keys, foreign keys, and relationships

CODE AND OUTPUT :

```
AI-16.1sql>...
1  --create tables Students, Courses, Enrollments, primary keys, foreign keys, and relationships
2  CREATE TABLE Students (
3      student_id INT PRIMARY KEY,
4      first_name VARCHAR(50),
5      last_name VARCHAR(50),
6      email VARCHAR(100)
7  );
8  CREATE TABLE Courses (
9      course_id INT PRIMARY KEY,
10     course_name VARCHAR(100),
11     course_code VARCHAR(10)
12 );
13 CREATE TABLE Enrollments (
14     enrollment_id INT PRIMARY KEY,
15     student_id INT,
16     course_id INT,
17     enrollment_date DATE,
18     FOREIGN KEY (student_id) REFERENCES Students(student_id),
19     FOREIGN KEY (course_id) REFERENCES Courses(course_id)
20 );
21 --insert sample data into Students, Courses, Enrollments tables
22 INSERT INTO Students (student_id, first_name, last_name, email) VALUES
23 (1, 'John', 'Doe', 'john.doe@gmail.com'),
24 (2, 'Jane', 'Smith', 'janesmith@gmail.com'),
25 (3, 'Michael', 'Johnson', 'michaeljohnson@gmail.com');
26 INSERT INTO Courses (course_id, course_name, course_code) VALUES
27 (1, 'Introduction to Computer Science', 'CS101'),
28 (2, 'Data Structures and Algorithms', 'CS102'),
29 (3, 'Database Systems', 'CS103');
30 INSERT INTO Enrollments (enrollment_id, student_id, course_id, enrollment_date) VALUES
31 (1, 1, 1, '2024-01-15'),
32 (2, 1, 2, '2024-01-16'),
33 (3, 2, 1, '2024-01-17'),
34 (4, 3, 3, '2024-01-18');
35 --query to retrieve all courses enrolled by a student
36 SELECT s.first_name, s.last_name, c.course_name
37 FROM Students s
38 JOIN Enrollments e ON s.student_id = e.student_id
39 JOIN Courses c ON e.course_id = c.course_id
40 WHERE s.student_id = 1;
41 --query to count number of students in each course
42 SELECT c.course_name, COUNT(e.student_id) AS student_count
43 FROM Courses c
44 LEFT JOIN Enrollments e ON c.course_id = e.course_id
45 GROUP BY c.course_name;
```

SQL v < 1 / 1 > 1 - 0 of 0

SQL v < 1 / 1 > 1 - 0 of 0

SQL v < 1 / 1 > 1 - 0 of 0

SQL v < 1 / 1 > 1 - 0 of 0

SQL v < 1 / 1 > 1 - 0 of 0

SQL v < 1 / 1 > 1 - 0 of 0

SQL v < 1 / 1 > 1 - 2 of 2

first_name	last_name	course_name
John	Doe	Introduction to Computer Science
John	Doe	Data Structures and Algorithms

SQL v < 1 / 1 > 1 - 3 of 3

course_name	student_count
Data Structures and Algorithms	1
Database Systems	1
Introduction to Computer Science	2

JUSTIFICATION :

This task helped me understand how student and course data are stored in a database. I learned how to connect tables using primary key and foreign key. It also improved my knowledge of writing SQL queries to get useful information.

Task 2 – Hospital Management Database

PROMPT :

--create tables doctors, patients, appointments, define constraints(unique IDs,valid dates)

CODE AND OUTPUT :

```

46
47
48 --create tables doctors, patients, appointments, define constraints(unique IDs,valid dates)
49 CREATE TABLE Doctors (
50     doctor_id INT PRIMARY KEY,
51     first_name VARCHAR(50),
52     last_name VARCHAR(50),
53     specialty VARCHAR(100)
54 );
55 CREATE TABLE Patients (
56     patient_id INT PRIMARY KEY,
57     first_name VARCHAR(50),
58     last_name VARCHAR(50),
59     date_of_birth DATE
60 );
61 CREATE TABLE Appointments (
62     appointment_id INT PRIMARY KEY,
63     doctor_id INT,
64     patient_id INT,
65     appointment_date DATE,
66     FOREIGN KEY (doctor_id) REFERENCES Doctors(doctor_id),
67     FOREIGN KEY (patient_id) REFERENCES Patients(patient_id)
68 );
69 --insert sample data into doctors, patients, appointments tables
70 INSERT INTO Doctors (doctor_id, first_name, last_name, specialty) VALUES
71 (1, 'Dr. Smith', 'Johnson', 'Cardiology'),
72 (2, 'Dr. Emily', 'Davis', 'Pediatrics'),
73 (3, 'Dr. Michael', 'Brown', 'Orthopedics');
74 INSERT INTO Patients (patient_id, first_name, last_name, date_of_birth) VALUES
75 (1, 'John', 'Doe', '1980-05-15'),
76 (2, 'Jane', 'Smith', '1990-08-20'),
77 (3, 'Michael', 'Johnson', '1975-12-10');
78 INSERT INTO Appointments (appointment_id, doctor_id, patient_id, appointment_date) VALUES
79 (1, 1, 1, '2024-06-01'),
80 (2, 1, 2, '2024-06-02'),
81 (3, 2, 1, '2024-06-03'),
82 (4, 3, 1, '2024-06-04');
83 -- list all appointments for a specific doctor
84 SELECT d.first_name, d.last_name, p.first_name, p.last_name, a.appointment_date
85 FROM Doctors d
86 JOIN Appointments a ON d.doctor_id = a.doctor_id
87 JOIN Patients p ON a.patient_id = p.patient_id
88 WHERE d.doctor_id = 1;
89 --retrieve patient history by patient ID
90 SELECT p.first_name, p.last_name, a.appointment_date, d.first_name, d.last_name, d.specialty
91 FROM Patients p
92 JOIN Appointments a ON p.patient_id = a.patient_id
93 JOIN Doctors d ON a.doctor_id = d.doctor_id
94 WHERE p.patient_id = 1;
95 --count total patients treated by each doctor
96 SELECT d.first_name, d.last_name, COUNT(a.patient_id) AS patient_count
97 FROM Doctors d
98 LEFT JOIN Appointments a ON d.doctor_id = a.doctor_id
99 GROUP BY d.doctor_id;
100
101
102

```

SQL > < 1 / 1 > 1 - 0 of 0

SQL > < 1 / 1 > 1 - 0 of 0

SQL > < 1 / 1 > 1 - 0 of 0

SQL > < 1 / 1 > 1 - 0 of 0

SQL > < 1 / 1 > 1 - 0 of 0

SQL > < 1 / 1 > 1 - 0 of 0

SQL > < 1 / 1 > 1 - 2 of 2

first_name	last_name	first_name	last_name	appointment_date
Dr. Smith	Johnson	John	Doe	2024-06-01
Dr. Smith	Johnson	Jane	Smith	2024-06-02

SQL > < 1 / 1 > 1 - 2 of 2

first_name	last_name	appointment_date	first_name	last_name	specialty
John	Doe	2024-06-01	Dr. Smith	Johnson	Cardiology
John	Doe	2024-06-04	Dr. Michael	Brown	Orthopedics

SQL > < 1 / 1 > 1 - 3 of 3

first_name	last_name	patient_count
Dr. Smith	Johnson	2
Dr. Emily	Davis	1
Dr. Michael	Brown	1

JUSTIFICATION :

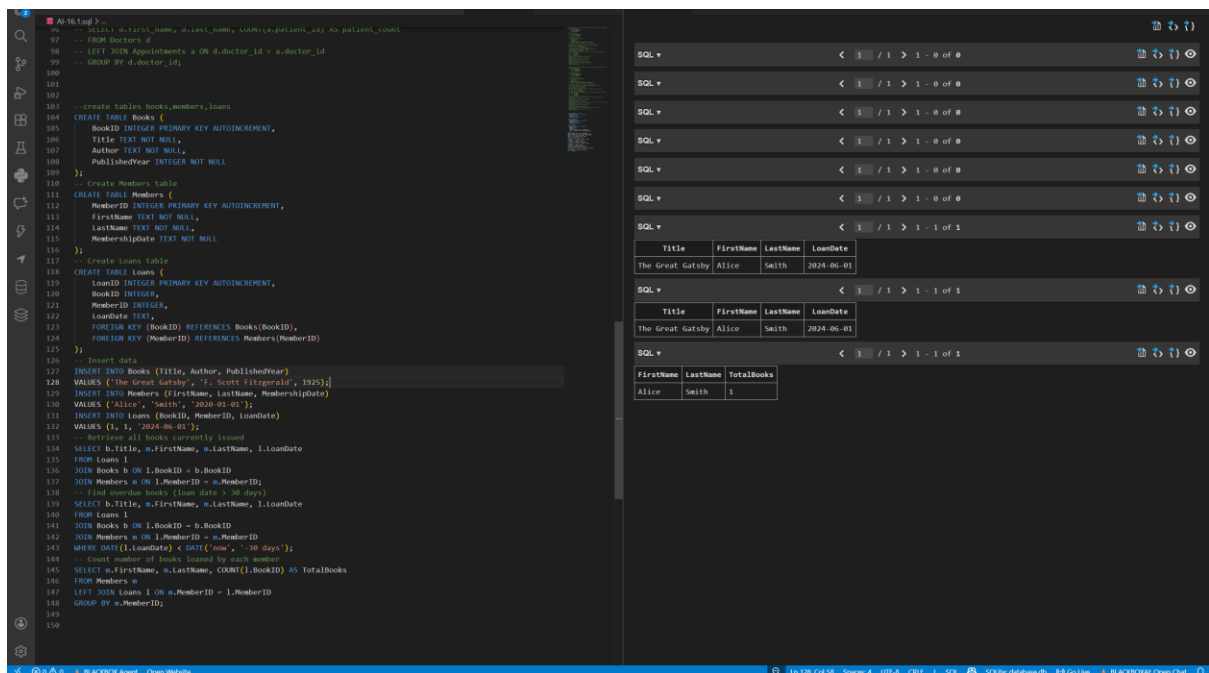
In this task, I understood how databases are used in hospitals to manage doctors, patients, and appointments. I learned about using constraints to avoid wrong data. It also helped me practice queries for real-life situations.

Task 3 – Library Database

PROMPT :

--create tables books,members,loans

CODE AND OUTPUT :



JUSTIFICATION :

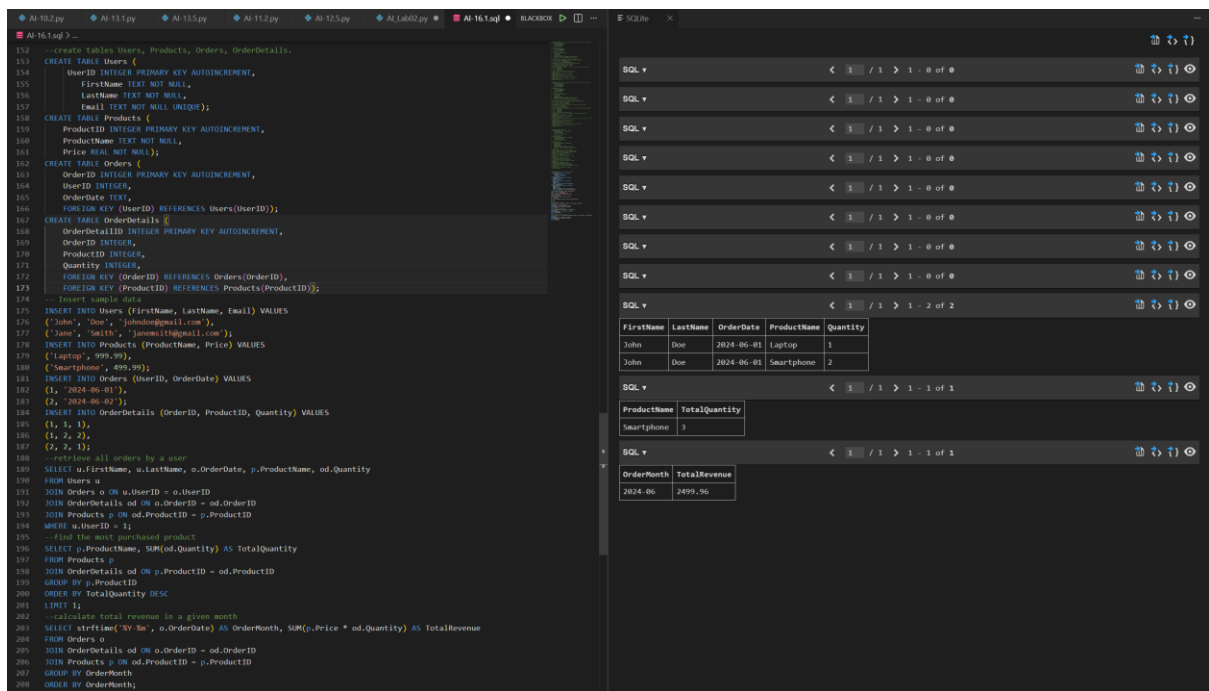
This task helped me learn how a library system works using a database. I understood how to track books and members. I also learned how indexing improves performance and how to find overdue books using queries.

Task 4 – Real-Time Application: Online Shopping Database

PROMPT :

--create tables Users, Products, Orders, OrderDetails.

CODE AND OUTPUT :



JUSTIFICATION :

This task gave me an idea about how e-commerce websites store data. I learned how different tables like users, products, and orders are connected. Writing queries like total revenue helped me understand business analysis.

Task 5 – University Examination Database

PROMPT :

--create tables Student,Subjects,Exams,Results.primary keys, foreign keys, referential integrity constraints

CODE AND OUTPUT :

AI 161mg > ...

```
--create tables Student,Subjects,Exams,Results,primary keys, foreign keys, referential integrity constraints
212 CREATE TABLE Student (
213     StudentID INTEGER PRIMARY KEY AUTOINCREMENT,
214     FirstName TEXT NOT NULL,
215     LastName TEXT NOT NULL,
216     DateOfBirth DATE NOT NULL
217 );
218
219 CREATE TABLE Subjects (
220     SubjectID INTEGER PRIMARY KEY AUTOINCREMENT,
221     SubjectName TEXT NOT NULL
222 );
223
224 CREATE TABLE Exams (
225     ExamID INTEGER PRIMARY KEY AUTOINCREMENT,
226     SubjectID INTEGER,
227     ExamDate DATE NOT NULL,
228     FOREIGN KEY (SubjectID) REFERENCES Subjects(SubjectID)
229 );
230
231 CREATE TABLE Results (
232     ResultID INTEGER PRIMARY KEY AUTOINCREMENT,
233     StudentID INTEGER,
234     ExamID INTEGER,
235     Score INTEGER NOT NULL,
236     FOREIGN KEY (StudentID) REFERENCES Student(StudentID),
237     FOREIGN KEY (ExamID) REFERENCES Exams(ExamID)
238 );
239
240 --insert sample data into Student, Subjects, Exams, Results tables
241 INSERT INTO Student (FirstName, LastName, DateOfBirth) VALUES
242 ('John', 'Doe', '2000-01-01'),
243 ('Jane', 'Smith', '2001-02-02');
244
245 INSERT INTO Subjects (SubjectName) VALUES
246 ('Mathematics'),
247 ('Science');
248
249 INSERT INTO Exams (SubjectID, ExamDate) VALUES
250 (1, '2024-06-01'),
251 (2, '2024-06-02');
252
253 --insert examination results for a student
254 SELECT s.FirstName, s.LastName, sub.SubjectName, r.Score
255 FROM Student s
256 JOIN Results r ON s.StudentID = r.StudentID
257 JOIN Exams e ON r.ExamID = e.ExamID
258 JOIN Subjects sub ON e.SubjectID = sub.SubjectID
259 WHERE s.StudentID = 1;
260
261 --calculate average marks for each subject
262 SELECT sub.SubjectName, AVG(r.Score) AS AverageScore
263 FROM Subjects sub
264 JOIN Exams e ON sub.SubjectID = e.SubjectID
265 JOIN Results r ON e.ExamID = r.ExamID
266 GROUP BY sub.SubjectID;
```

SQL v

< 1 / 1 > 1 - 0 of 0

SQL v

< 1 / 1 > 1 - 0 of 0

SQL v

< 1 / 1 > 1 - 0 of 0

SQL v

< 1 / 1 > 1 - 0 of 0

SQL v

< 1 / 1 > 1 - 0 of 0

SQL v

< 1 / 1 > 1 - 0 of 0

SQL v

< 1 / 1 > 1 - 0 of 0

SQL v

< 1 / 1 > 1 - 2 of 2

FirstName	LastName	SubjectName	Score
John	Doe	Mathematics	85
John	Doe	Science	90

SQL v

< 1 / 1 > 1 - 2 of 2

SubjectName	AverageScore
Mathematics	81.5
Science	89.0

JUSTIFICATION :

This task helped me understand how exam data is managed. I learned how to store marks and calculate averages values using SQL. It also improved my skills in using joins and aggregation functions.